

A vertical bar on the left side of the page with a gradient from dark green at the top to dark brown at the bottom.

BITEWISE

Platform Overview

What BiteWise does, the experience we built, the stack behind it, and how it uses Swiggy MCP.

BiteWise · Product & technical overview · July 26, 2026

CONTENTS

1. What is BiteWise

2. The Problem & What We're Trying to Achieve

3. Product 1 — NutriOrder AI

4. Product 2 — SmartPantry AI

5. Shared Platform Experience

6. Technology Stack

7. Swiggy MCP Integration

8. Security & Safety Engineering

9. Current Status

1. What is BiteWise

BiteWise is a food intelligence platform built on Swiggy MCP (Model Context Protocol), housing two focused, independently-onboarded products under one account:

- **NutriOrder AI** — a personal health guide that tracks a user's biometric goals and closes the gap to them through health-aware meal ordering.
- **SmartPantry AI** — a household kitchen intelligence system that tracks real pantry stock, matches it against recipes, flags what is low or expiring, and turns gaps into grocery orders.

Both products share one account, one design system, and one platform shell, but are deliberately built as two separate experiences rather than a single generic “food app” — each has its own onboarding, its own home screen, and its own reason a user opens it.

2. The Problem & What We're Trying to Achieve

Most food-delivery and grocery apps are search-first: open the app, type what you want, buy it. That works, but it treats every session as stateless — the app has no memory of your health goals and no idea what's already in your kitchen.

BiteWise's core bet is that **state should come before search**. A user should open NutriOrder AI and see their actual progress toward their goal before they're asked what to order. A user should open SmartPantry AI and see what's running low or about to expire before they're asked what to cook. Ordering and buying become the *mechanism* a user reaches for once the platform has told them something useful — not the opening question.

Concretely, that means:

- Every meal recommendation is ranked against a real, user-tunable set of priorities (protein, calories, budget, delivery speed, restaurant rating, macro-estimate confidence) — not a generic popularity sort.
 - Every recipe suggestion is scored against a household's *actual* pantry inventory, not a static “recipe of the day.”
 - Every order — food or grocery — goes through the same real Swiggy MCP tool calls a production integration would use, with the same safety rules a real deployment would need.
-

3. Product 1 — NutriOrder AI

3.1 Positioning

NutriOrder AI's home screen is a health **Coach**, not a search box. The philosophy: track first, order second. A five-step onboarding wizard (Welcome → Biometrics → Goals & Diet → Ranking → Confirm) establishes a real BMR/TDEE-driven calorie and protein target before a user ever sees a restaurant.

3.2 User Journey

1. **Onboarding** — biometrics (age, height, weight, activity level), fitness goal and diet preference, and a **Personalized Ranking Controls** step where the user sets six independent weights: Protein Focus, Calorie Compliance, Budget Friendly, Delivery Speed, Restaurant Rating, and Clean Eating (which penalizes low-confidence macro estimates). These weights are used directly in the ranking pipeline, not cosmetic.
2. **Coach (home)** — today's calorie/protein progress against target, a consistency streak, a 7-day trend chart, a weight-log widget, and a promoted “What do you need next?” card that computes the user's *remaining* calorie/protein gap for the day and re-runs the full recommendation pipeline against exactly that gap.
3. **Find Food (Order)** — address selection starts a real order session; nothing searches until the user explicitly submits a query or picks a suggested prompt. Results are ranked using the six priority weights plus the live gap from Coach. A full cart review (restaurant, payment method, coupons, safety banner) precedes an explicit confirmation checkbox before checkout is even enabled.
4. **History** — every order session, most recent first.

5. **Preferences** — the same biometrics and ranking controls from onboarding, editable anytime.

3.3 The Recommendation Engine

Recommendations run through a personalization pipeline (`agent/pipeline.py` , `agent/personalization.py`) that combines the user's biometric targets, their six ranking weights, and (from the Coach hub) their real-time remaining macro gap, then queries Swiggy Food MCP for live menu data and re-ranks the results. Each result surfaces a match score and plain-language reasoning, not just a number.

4. Product 2 — SmartPantry AI

4.1 Positioning

SmartPantry AI's home screen is a pantry-state hub, not a “what do you want to cook” search box. The same track-first philosophy applies: a brand-new pantry has nothing to be intelligent about, so onboarding exists specifically to seed real state before the home screen has anything to show.

4.2 User Journey

1. **Onboarding** — a category-grouped stock checklist (Staples, Dairy, Proteins, Vegetables, Spices), checked by default, seeds real pantry rows the moment the user continues; a cook-query step immediately checks a named dish against what was just stocked (coverage %, missing items) as the first real payoff; an optional household step adds family members with their own diet/allergies.
2. **Kitchen (home)** — low-stock and expiring-soon alerts, computed from real pantry rows and per-category default shelf lives, sit above “What Can I Cook Today?” — every one of 30 recipe templates scored against actual stock, boosted toward using soon-expiring ingredients first, filtered by household dietary constraints. The free-text composer for ad-hoc cravings or grocery lists sits below all of that, as the action a user reaches for, not the opener.
3. **Pantry** — the qualitative ground truth (`full` / `half` / `low` / `empty` stock levels, not exact quantities, by deliberate design to keep logging fast) that every other page reads from.
4. **Grocery** — a shopping list that groups items by category and urgency, and feeds a real Instamart cart-preview → checkout flow. Purchasing an item automatically restocks the matching pantry row back to full, closing the loop.
5. **Household** — family members with their own diet, allergies, and nutrition targets; combined into a household-wide insight view and used to filter recipe suggestions everywhere in the product.

4.3 The Intelligence Layer

SmartPantry's recipe and stock logic (`backend/household/intelligence.py`) is explicitly **deterministic and rule-based — no LLM calls** — by design, for predictability and speed:

- **Low-stock detection** — flags `low` / `empty` items, auto-adds `empty` items to the grocery list.
 - **Cook-today matching** — real pantry-vs-recipe coverage scoring against 30 recipe templates, boosted by expiring-ingredient use, filtered by household diet/allergy constraints.
 - **Expiry tracking** — category-based default shelf lives (Dairy 4 days, Proteins 4 days, Bakery 4 days, Vegetables 5 days) when no explicit date is set, with an alert window tuned to stay above those defaults so a freshly-stocked item never reads as already expiring.
 - **Grocery grouping** — buckets the shopping list by category and assigns urgency based on whether an item matches a low-stock pantry row or has sat unpurchased for several days.
 - **Auto-restock on purchase** — checking off a grocery item, or completing an Instamart checkout, resets the matching pantry item to `full` automatically.
-

5. Shared Platform Experience

- **One-time platform chooser** — a first-time-only “What are you here for today?” screen; returning visits skip straight to whichever product was last used, tracked via a lightweight local flag.
 - **Persistent sidebar** — a workspace-style product switcher plus each product's own sub-navigation, shared across both products via one design system.
 - **Light/dark theme system** — a full semantic design-token palette (`bg` , `surface` , `nutri` , `pantry` , `brand` , status colors) with a theme toggle that persists across reloads.
 - **Onboarding-aware navigation** — sidebar links are visibly disabled while a product's onboarding is still in progress, rather than allowing a click that changes the URL without changing what's on screen.
-

6. Technology Stack

Frontend

- Next.js 16.2.9 (App Router)
- React 19.2.4
- TypeScript 5
- Tailwind CSS v4 (semantic design-token system)
- Framer Motion 12 (landing-page motion)
- Playwright-driven regression suite (dozens of end-to-end checks covering both products' full flows)

Backend

- FastAPI (Python)
- SQLAlchemy 2.0 (ORM)
- Pydantic 2 (schema validation)
- cryptography (Fernet-based encryption for stored OAuth tokens)
- SQLite (development datastore)

7. Swiggy MCP Integration

BiteWise integrates directly against the **Swiggy Builders Club MCP servers** — the Food server and the Instamart server — verified against the official reference documentation at `mcp.swiggy.com/builders`.

7.1 Swiggy Food MCP

The Food server exposes **14 tools** across five categories (Discover, Cart, Order, Track, Support). BiteWise wraps **12 of the 14**:

Category	Tools used
Discover	<code>get_addresses</code> , <code>search_restaurants</code> , <code>search_menu</code> , <code>get_restaurant_menu</code>
Cart	<code>get_food_cart</code> , <code>update_food_cart</code> , <code>fetch_food_coupons</code> , <code>apply_food_coupon</code> , <code>flush_food_cart</code>
Order	<code>place_food_order</code>
Track	<code>get_food_orders</code> , <code>track_food_order</code>

(`get_food_order_details` and the Support-category `report_error` tool are documented but not currently wired into the product.)

7.2 Swiggy Instamart MCP

The Instamart server exposes **13 tools** across the same five categories. BiteWise wraps **5 of the 13**: `search_products` , `update_cart` , `get_cart` , `checkout` , `get_orders` .

7.3 Authentication & Safety Gating

- OAuth access tokens are stored encrypted at rest and decrypted only at call time.

- Three independent, environment-driven locks gate every real Swiggy call: `USE MOCK MCP` (defaults to true — routes to a local mock implementation instead of the live server), `SWIGGY_ENV` (defaults to "mock"), and `ALLOW_PLACE_ORDER` (defaults to `false` — a final, explicit switch specifically for the order-placement/checkout calls). All three must be deliberately configured for a real order to reach Swiggy's staging or production endpoints.
 - **Hard order-value caps**, matching the Swiggy Builders Club rules: Food orders are blocked outright at or above ₹1000; Instamart orders must fall between ₹99 and ₹1000. Both are enforced server-side, not just hidden in the UI.
 - Every checkout — food or grocery — requires an explicit user confirmation checkbox before the place-order action is even clickable.
-

8. Security & Safety Engineering

- **Tamper-evident sessions** — session tokens are HMAC-SHA256 signed (`user_id.signature`) and verified on every request; a forged or altered token is rejected.
 - **Encrypted credential storage** — Swiggy OAuth tokens are encrypted before being persisted.
 - **Proxy-aware rate limiting** — mutating endpoints are rate-limited per client, resolving the real client IP from `X-Forwarded-For` so the limiter works correctly behind a reverse proxy.
 - **Explicit confirmation everywhere it matters** — no order, food or grocery, is ever placed without a user actively checking a confirmation box first.
-

9. Current Status

BiteWise runs in **mock MCP mode by default** — every Swiggy tool call resolves against a local mock implementation that mirrors the real servers' response shapes, so the full product experience (search, ranking, cart, checkout, tracking) works end-to-end without live credentials. This is a deliberate default for development and demonstration; connecting real Swiggy staging or production credentials and flipping the three safety locks is the only step required to move a deployment from mock to live.

The full Playwright regression suite (both products' onboarding, tracking, and ordering flows, theme persistence, and returning-user routing) passes end-to-end against this mock-mode build, alongside a backend `pytest` suite covering the coach, pantry, household, and order domains.